

EQ2341 Final Project

Activity Recognition Using a Baum-Welch-Trained Hidden Markov Model

Oguzhan Oguz
oguzhano@kth.se

1 Introduction

The objective of this project is to implement a classifier for human activity recognition using phone accelerometer data. The target activities are standing still, walking, and running. This task is naturally sequential: a person normally remains in the same activity for several consecutive time instants, and sudden isolated label changes are unlikely. A Hidden Markov Model (HMM) is therefore a suitable classifier because it combines observation likelihoods with temporal state dynamics.

The implemented system uses phone accelerometer logs, extracts window-level motion features, trains a three-state Gaussian-emission HMM, and decodes the activity sequence using both Forward-Backward posterior probabilities and the Viterbi algorithm. The HMM parameters are initialized from labeled single-activity recordings and then refined using the Baum-Welch expectation-maximization (EM) algorithm.

2 Dataset and recording protocol

The data were recorded using a phone accelerometer. Each raw sample contains a timestamp and three acceleration components,

$$\mathbf{x}_t = [a_x(t) \quad a_y(t) \quad a_z(t)]^T.$$

Three separate recordings were collected for each single activity: standing, walking, and running. These nine recordings were used for model training. One additional mixed recording was collected for testing. The mixed recording was performed in the order

standing $\rightarrow$ walking $\rightarrow$ running $\rightarrow$ walking $\rightarrow$ standing $\rightarrow$ running.

Table 1: Dataset summary.

Purpose	Files	Use
Training	3 standing recordings	Estimate standing emission statistics
Training	3 walking recordings	Estimate walking emission statistics
Training	3 running recordings	Estimate running emission statistics
Testing	1 mixed recording	Evaluate sequential classification

The recordings are realistic phone sensor data and include small artifacts caused by starting the recording, placing the phone in the pocket, removing it, and stopping the recording. To reduce the effect of these artifacts, the first and last 8 seconds of each single-activity training recording were removed before training.

3 Preprocessing and feature extraction

The raw acceleration vector was first converted into acceleration magnitude,

$$m_t = \sqrt{a_x(t)^2 + a_y(t)^2 + a_z(t)^2}.$$

A 5-sample moving average was applied to the magnitude sequence to reduce high-frequency measurement noise. Since the sampling rate was approximately 125 Hz, this smoothing length is short compared with the 1-second analysis window and therefore does not remove the main walking or running motion pattern. The classifier then used overlapping windows with a 1-second window length and a 0.5-second hop size. For each window, three features were extracted:

$$\mathbf{z}_n = \begin{bmatrix} \mu_m(n) \\ \sigma_m(n) \\ r_m(n) \end{bmatrix},$$

where $\mu_m(n)$ is the mean magnitude, $\sigma_m(n)$ is the standard deviation of magnitude, and $r_m(n)$ is the range of the magnitude values inside the window. After window-level feature extraction, each feature dimension was standardized using the mean and standard deviation estimated from the training set only. The same training-set normalization constants were then applied to the mixed test recording.

Magnitude-based features make the classifier more robust to phone orientation because they do not depend directly on the coordinate axes. However, this is also a design trade-off: axis-specific directional motion patterns are discarded.

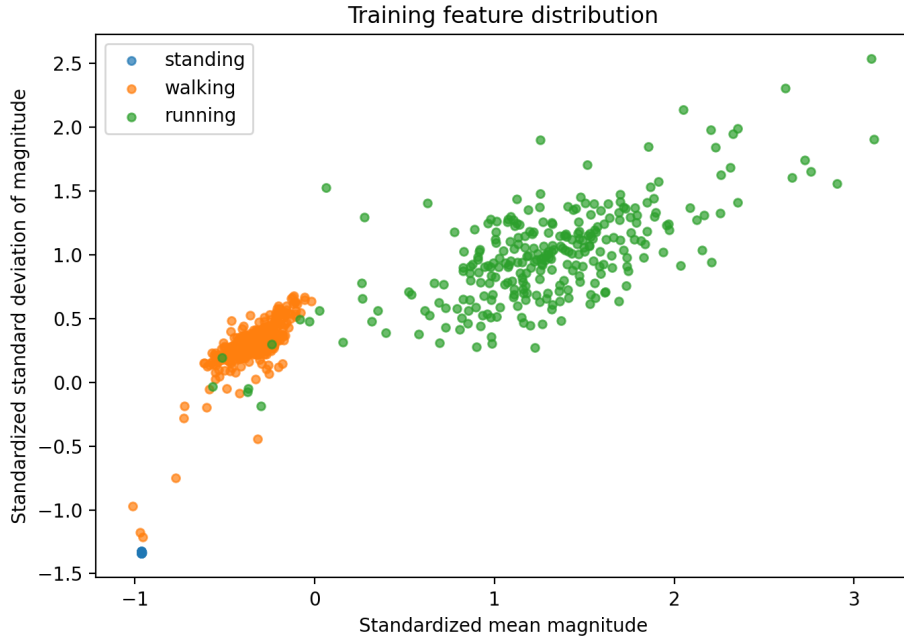


Figure 1: Training feature distribution using standardized mean magnitude and standardized standard deviation of magnitude.

4 HMM classifier

The raw accelerometer samples are first converted into window-level feature vectors $\mathbf{z}_n$. The HMM state S_n then represents the activity associated with window n :

$$S_n \in \{\text{standing, walking, running}\}.$$

Each state uses a multivariate Gaussian output distribution over the three-dimensional window feature vector:

$$p(\mathbf{z}_n | S_n = j) = \mathcal{N}(\mathbf{z}_n; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j).$$

The HMM is specified by the initial probability vector $\mathbf{q}$, the transition matrix $\mathbf{A}$, and the Gaussian emission parameters $\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j$.

4.1 Initialization

The training recordings are labeled at the file level, so the model was first initialized using the known activity labels. This produces stable initial estimates of the state emission means, covariance matrices, and transition probabilities. This supervised initialization is used only as a starting point for the Baum-Welch refinement, not as the final training algorithm.

Because the training recordings are single-activity sequences, they mainly contain self-transitions. Therefore, the transition matrix is best interpreted as a temporal persistence model. The off-diagonal transition probabilities are only weakly informed by actual activity changes and are partly controlled by pseudocount smoothing and Baum-Welch refinement. A more rigorous dataset would include accurately labeled transition recordings.

4.2 Baum-Welch EM training

After initialization, the HMM was refined using Baum-Welch EM. For notational simplicity, the update equations below are written for one observation sequence of length T . In the implementation, the sufficient statistics were accumulated over all training recordings before each M-step update.

In the E-step, Forward-Backward inference computes the posterior state probabilities

$$\gamma_j(n) = P(S_n = j | \mathbf{Z}, \lambda)$$

and the posterior transition probabilities

$$\xi_{ij}(n) = P(S_n = i, S_{n+1} = j | \mathbf{Z}, \lambda).$$

In the M-step, the initial probabilities, transition probabilities, and Gaussian parameters are updated using these posterior weights:

$$\begin{aligned} q_j^{\text{new}} &= \gamma_j(1), \\ A_{ij}^{\text{new}} &= \frac{\sum_{n=1}^{T-1} \xi_{ij}(n)}{\sum_{n=1}^{T-1} \gamma_i(n)}, \\ \boldsymbol{\mu}_j^{\text{new}} &= \frac{\sum_{n=1}^T \gamma_j(n) \mathbf{z}_n}{\sum_{n=1}^T \gamma_j(n)}, \\ \boldsymbol{\Sigma}_j^{\text{new}} &= \frac{\sum_{n=1}^T \gamma_j(n) (\mathbf{z}_n - \boldsymbol{\mu}_j^{\text{new}})(\mathbf{z}_n - \boldsymbol{\mu}_j^{\text{new}})^T}{\sum_{n=1}^T \gamma_j(n)}. \end{aligned}$$

The implementation uses a convergence criterion based on the absolute training log-likelihood improvement:

$$|\mathcal{L}^{(k)} - \mathcal{L}^{(k-1)}| < 10^{-3}.$$

The final log-likelihood is computed after the final parameter update.

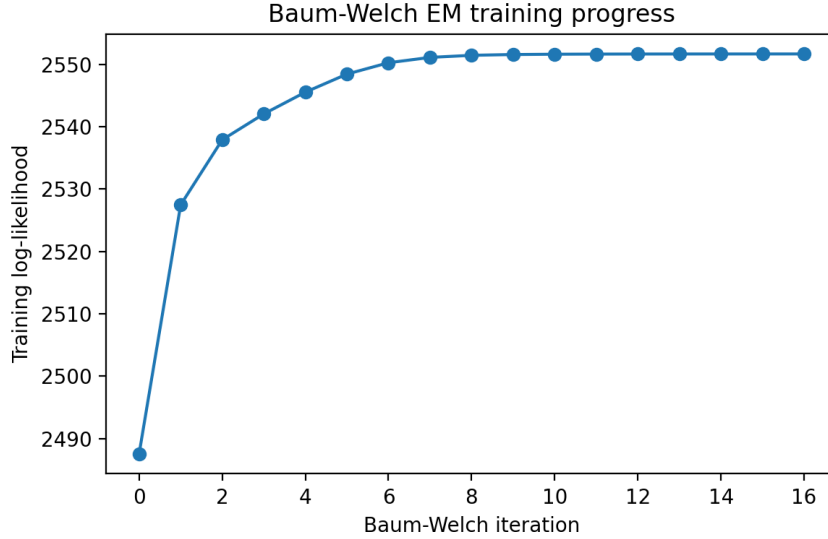


Figure 2: Baum-Welch training log-likelihood. The log-likelihood increases from 2487.54 to 2551.70 and stops when the improvement falls below the convergence threshold.

Table 2: Final Baum-Welch transition matrix. Rows correspond to the current state and columns correspond to the next state.

	standing	walking	running
standing	0.9964	0.0018	0.0018
walking	0.0017	0.9919	0.0064
running	0.0020	0.0030	0.9950

The final transition matrix is strongly diagonal, which is consistent with the activity-recognition setting: each activity tends to persist over several consecutive windows. Since the training recordings are single-activity recordings, the off-diagonal entries should mainly be interpreted as small transition allowances rather than as a fully data-driven model of real human activity switching frequencies.

5 Evaluation protocol

The model was trained on the nine single-activity recordings and evaluated on the mixed test recording. Exact transition timestamps were not explicitly measured in the notebook. Therefore, the mixed sequence was divided into six approximate equal-duration segments according to the known activity order. To reduce ambiguity near transitions, the first and last 5 seconds of the mixed sequence and a 3-second margin around each approximate segment boundary were excluded from the quantitative accuracy computation.

The reported accuracy is therefore an approximate overlapping-window-level accuracy. Because the windows overlap, the 167 scored windows should not be interpreted as 167 fully independent test examples. Overlapping windows make the decoded trajectory smoother, but reduce the effective number of independent samples.

Three decoding/classification methods were evaluated:

1. Forward-Backward posterior MAP decoding:

$$\hat{S}_n = \arg \max_j P(S_n = j \mid \mathbf{Z}, \lambda).$$

2. Viterbi decoding, which estimates the most probable complete state sequence.
3. An independent Gaussian baseline, which ignores temporal transitions and classifies each window using only the Gaussian emission likelihoods.

6 Experimental results

Table 3: Mixed-test classification results. The accuracy is approximate because the mixed ground-truth boundaries are approximate.

Method	Approx. accuracy	Scored windows
Forward-Backward posterior MAP	84.43%	167
Viterbi	85.03%	167
Independent Gaussian baseline	82.63%	167

The HMM-based methods outperform the independent Gaussian baseline. This suggests that the transition model and sequence decoding provide useful temporal smoothing compared with classifying each window independently.

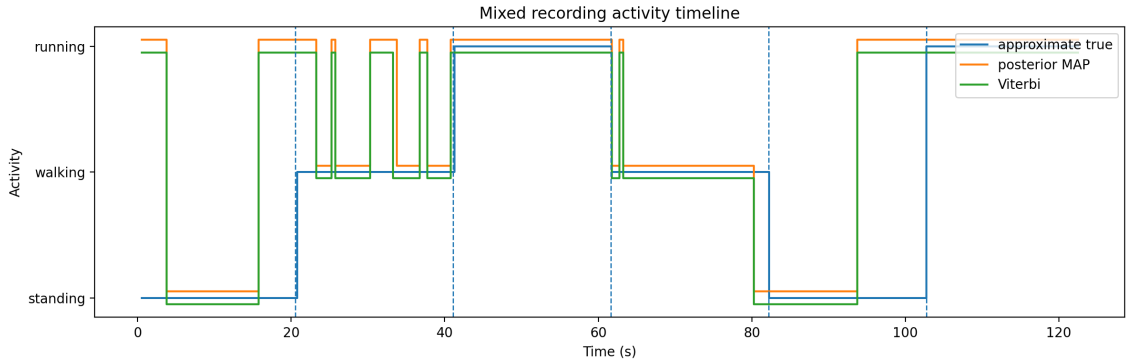


Figure 3: Approximate true activity labels and HMM predictions on the mixed recording. Dashed vertical lines show the approximate equal-duration segment boundaries.

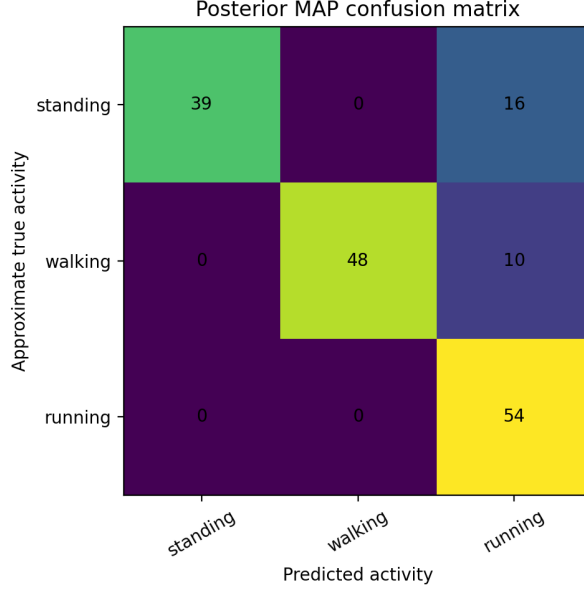


Figure 4: Confusion matrix for Forward-Backward posterior MAP decoding.

The posterior MAP confusion matrix is

Table 4: Posterior MAP confusion matrix. Rows are approximate true labels and columns are predicted labels.

	standing	walking	running
standing	39	0	16
walking	0	48	10
running	0	0	54

Table 5: Viterbi confusion matrix. Rows are approximate true labels and columns are predicted labels.

	standing	walking	running
standing	39	0	16
walking	0	49	9
running	0	0	54

Viterbi gives the best overall result in Table 3, with one additional walking window classified correctly compared with posterior MAP decoding. The posterior MAP confusion matrix is still included because posterior MAP is the direct output of the Forward-Backward posterior probabilities, while the Viterbi confusion matrix above shows the best complete-sequence decoding result.

Running is recognized very reliably, while most errors involve standing or walking windows being predicted as running. This should be interpreted carefully. Standing and running have clearly different motion-variation patterns in the training features, so part of this apparent confusion may plausibly come from imperfect temporal alignment between the real activity transitions and the approximate equal-duration ground-truth labels. The errors are therefore not necessarily only due to weak Gaussian emission modeling.

7 State identity after Baum-Welch

Because Baum-Welch is an unsupervised EM refinement step, HMM state labels are theoretically exchangeable. To ensure that the activity interpretation remained valid, the learned emission means were inspected after training. The standardized standard-deviation and range features remained ordered consistently: standing had the lowest motion-variation features, walking had intermediate values, and running had the highest values. Thus, the state identities remained consistent with the initialized activity labels.

Table 6: Learned standardized emission means after Baum-Welch.

State	mean magnitude	std magnitude	range magnitude
standing	-0.964	-1.335	-1.357
walking	-0.327	0.325	0.435
running	1.247	0.952	0.864

8 Limitations

The main limitation is the construction of the mixed-sequence ground truth. Since exact transition times were not recorded, the mixed sequence was labeled using approximate equal-duration segments. The reported accuracy is therefore useful as a project-level demonstration, but it should not be interpreted as a precise benchmark number.

The test set contains only one mixed recording from the same phone-placement setup. The dataset is also small: three recordings per single activity and one mixed test sequence. A stronger evaluation would include several mixed recordings, multiple users, multiple phone placements, different walking and running speeds, and explicit transition timestamps.

Another limitation is the use of a single Gaussian emission distribution for each activity. This gives a simple and interpretable model, but it may not capture more complex or multi-modal motion patterns that could appear with different users, phone placements, or activity speeds.

Finally, the transition matrix is learned mostly from single-activity recordings. It models temporal persistence well, but it should not be over-interpreted as a complete data-driven model of real human activity transitions.

9 Conclusion

This project implemented a complete HMM-based activity recognition classifier using phone accelerometer data. The system uses magnitude-based window features, Gaussian emissions, supervised initialization, Baum-Welch EM training, Forward-Backward posterior decoding, and Viterbi decoding. The HMM achieved approximately 84.43% posterior MAP accuracy and 85.03% Viterbi accuracy on the mixed test sequence, outperforming a non-temporal Gaussian baseline. The result demonstrates that an HMM can effectively exploit temporal structure for activity recognition, while also highlighting the importance of accurate transition labeling and larger test sets for more rigorous evaluation.